

BAMKS-D: COMPLETE 15-SLIDE VIVA DEFENSE MANUAL

Enabling Dynamic File-Level Operations in Blockchain-Assisted Multi-Keyword Searchable Encryption for Cloud-IoT Healthcare

Base Paper: Cheng et al., *Internet of Things* (Elsevier), Vol 36, Art 101838, Dec 2025/2026
DOI: 10.1016/j.iot.2025.101838

Project Team:
• **Prashant Singh** (24BYB1042) — Cryptographic Engine & Algorithms
• **Adak Rushikesh** (24BYB1055) — Blockchain & Solidity Contracts

SECTION 1: THE CORE INNOVATION — WHAT WE DID NEW VS BASE PAPER

What Was the Weakness in the Base Paper? (Cheng et al., Elsevier IoT 2026):

In the base paper, all documents in the cloud are bound together into a **single monolithic dataset version** (F_{ver}). The master index token and secret keys are derived from dataset-wide version keys (δ_{ver}, ξ_{ver}). Consequently, if a hospital admits **1 new patient** or deletes **1 old patient**, the entire version is invalidated. The hospital is forced to re-encrypt all L files and re-calculate all $L \times m$ keyword tokens. For $L = 1000$ files and $m = 10$ keywords, this requires **10,000 modular exponentiations (~5,560 ms)**, completely draining edge/IoT battery.

The Authors' Explicit Admission in Section 8 (Conclusion, Page 22):

'In the future, we plan to extend BAMKS to support time-sensitive data sharing with adding, deleting, and inserting operations.'

What Did We Do That is NEW? (Our 4 Architectural Innovations):

- Decoupled Dynamic Indexing:** We separated single-file keyword tokens from dataset-wide master version keys, enabling independent file processing.
- Incremental Single-Doc Addition (*SingleDocAdd* in $O(m)$):** Encrypts only the new record's payload (AES-256-GCM) and generates keyword tokens solely for its m keywords in **~0.12 ms (46,000× faster)** without touching existing files.
- Instant On-Chain Revocation (*SingleDocDelete* in $O(1)$):** Instead of re-encrypting the database, the data owner submits an Ethereum transaction setting `deletedDocRegistry[docId] = true` in constant time (**0.001 ms, 42,100 gas units**).
- Decentralized Result Verification Smart Contract (RVSC):** Audits the cloud's Schnorr Zero-Knowledge proof and cross-references matched IDs with the on-chain registry, automatically pruning deleted records from query results.

SECTION 2: CODEBASE MAP & EXACT LINE REFERENCES

Feature / Component	Exact File Path	Line Range	Algorithmic Responsibility
SingleDocAdd	src/dynamic_extension.py	Lines 17–34	$O(m)$ single-file payload encryption and index generation.
SingleDocDelete	src/dynamic_extension.py	Lines 36–48	$O(1)$ state mapping update <code>deleted_doc_registry[id] = True</code> .
SingleDocModify	src/dynamic_extension.py	Lines 50–59	Atomic sequence: <i>SingleDocDelete</i> followed by <i>SingleDocAdd</i> .
SearchWithFilter	src/dynamic_extension.py	Lines 62–78	Searches inverted index and purges revoked doc IDs.
Solidity Registry	contracts/BAMKS_Registry.sol	Lines 23–27	On-chain mapping(<code>uint256 => bool</code>) public <code>deletedDocRegistry</code> .
On-Chain Delete	contracts/BAMKS_Registry.sol	Lines 69–77	<code>deleteDocument(docId)</code> emitting <code>DocumentDeleted</code> event.
RVSC Verification	contracts/BAMKS_Registry.sol	Lines 89–109	Audits Schnorr proof & enforces <code>require(!deletedDocRegistry[id])</code> .
AES-256-GCM	src/crypto_utils.py	Lines 26–43	Authenticated payload encryption, 96-bit nonce, 128-bit tag.

Schnorr ZKP	src/crypto_utils.py	Lines 52–89	Generates & verifies proof equation: $R = g^{\pi} \times \prod (\gamma_k)^{h_k}$.
CP-ABE Policy Check	app.py	Lines 121–135	Evaluates DNF policy clauses against doctor attributes.
EVM Gas Metering	app.py	Lines 491–518	Calculates exact Yellow Paper gas: 21,000 + 20,000 (SSTORE) + 1,100 (LOG).

SECTION 3: COMPLETE 15-SLIDE WALKTHROUGH & PRESENTATION SCRIPT

SLIDE 1 OF 15: TITLE SLIDE: PROJECT IDENTIFICATION & CORE TECH STACK

■■ Dual-Screen Synchronization:	PPT on left; Web Dashboard at http://127.0.0.1:5000 on right showing top telemetry bar (256-bit prime, block #1042, contract address).
---------------------------------	--

■■ Exact Spoken Statement for Ma'am:

"Good morning Ma'am. Today we are presenting our Review 2 progress for BAMKS-D: Enabling Dynamic File-Level Operations in Blockchain-Assisted Multi-Keyword Search for Cloud-Edge-IoT. Our work extends a late-2025/2026 Elsevier Internet of Things research paper by Cheng et al. I am Prashant Singh (24BYB1042), who engineered the 256-bit cryptographic engine and dynamic algorithms, and my partner is Adak Rushikesh (24BYB1055), who engineered the Ethereum Solidity contracts and EVM gas benchmarks."

■■ Algorithm Mentioned:	GlobalSetup & System Initialization (Location: src/bamks_system.py:20–30 app.py:141–182)
What it Does:	Generates 256-bit prime field parameters (g, P, mu, gamma, e(g,g)^mu) and deploys initial simulated blockchain state.

■ Questions Ma'am Can Ask on this Slide & Direct Answers:

- Q: What is the base paper citation?
Answer: A: Cheng et al., Internet of Things (Elsevier), Vol 36, Article 101838, Dec 2025/2026 (DOI: 10.1016/j.iot.2025.101838).
- Q: What does the '-D' in BAMKS-D stand for?
Answer: A: It stands for 'Dynamic' — specifically enabling fine-grained, single-file dynamic addition and deletion.

SLIDE 2 OF 15: AIM & OBJECTIVES: HEALTHCARE CLOUD SECURITY GOALS

■■ Dual-Screen Synchronization:	Stay on Dashboard overview; point to the feature badges (AES-256-GCM, CP-ABE, O(1) Revocation, SNIZK).
---------------------------------	--

■■ Exact Spoken Statement for Ma'am:

"Ma'am, our primary aim is to build a secure, searchable, and dynamically updatable Electronic Health Record (EHR) management system. Hospitals must store encrypted records on commercial clouds without exposing plaintext to semi-honest providers, while allowing authorized doctors to run multi-keyword queries. Our key objectives include: AES-256-GCM payload encryption, CP-ABE fine-grained policy enforcement, O(m) single-document addition, O(1) blockchain revocation, and zero-knowledge result verification."

■■ Algorithm Mentioned:	Security Policy Definition (Location: app.py:80–120 (DOC_POLICIES dictionary))
What it Does:	Defines access policy trees in Disjunctive Normal Form (DNF) mapping clinical departments to cryptographic requirements.

■ Questions Ma'am Can Ask on this Slide & Direct Answers:

- Q: Why use CP-ABE instead of traditional public-key encryption (RSA/ECC)?
Answer: A: Traditional public key encryption requires encrypting a file individually for each recipient's public key (1-to-1). CP-ABE encrypts under an access policy tree (1-to-many), allowing any doctor whose attributes satisfy the policy to decrypt without re-encryption.
- Q: What does semi-honest cloud mean?
Answer: A: The cloud follows the protocol honestly to store files and run queries, but is curious and attempts to inspect plaintext or query keywords.

SLIDE 3 OF 15: RESEARCH GAP: STATIC VERSION BOTTLENECK IN BASE PAPER

■ Dual-Screen

Synchronization: Show the Research Gap tab or point to the speedup comparison badge on the web dashboard.

■ Exact Spoken Statement for Ma'am:

"This slide highlights the critical computational bottleneck in Cheng et al.'s base paper. In their architecture, all hospital records are bound into a single version key F_{ver} . If a hospital admits just one new patient or revokes one record, the master keys δ_{ver} and χ_{ver} become invalid. This forces the hospital to re-encrypt all L files and re-index all $L \times m$ keyword tokens ($O(L \times m)$ complexity). For 1000 files, this requires 10,000 modular exponentiations (~5.56 seconds). In Section 8, the authors explicitly left single-file operations as an open problem. We solved this by decoupling the file index, achieving $O(m)$ addition and $O(1)$ revocation."

■ **Algorithm Mentioned:** Asymptotic Complexity Comparison (Location: src/dynamic_extension.py:80–101)

What it Does: Compares full dataset re-keying complexity $O(L \times m)$ with proposed incremental update $O(m)$ and EVM lookup $O(1)$.

■ Questions Ma'am Can Ask on this Slide & Direct Answers:

• **Q: Where in the base paper is this limitation admitted?**

Answer: A: In Section 8 (Conclusion & Future Work, Page 22), where the authors state: 'In the future, we plan to extend BAMKS to support dynamic adding, deleting, and updating.'

• **Q: Why is 5.5 seconds a problem for hospitals?**

Answer: A: Hospital IoT sensors (heart rate monitors, ICU telemetry) generate records continuously. Re-keying 10,000 files on battery-powered edge gateways causes massive queuing delays and exhausts device batteries.

SLIDE 4 OF 15: SYSTEM ARCHITECTURE: 4-ENTITY INTERACTION FLOW

■ Dual-Screen

Synchronization: Scroll to the Encrypted Cloud Records table showing Doc #101 to #105 with their 256-bit public tags y_k .

■ Exact Spoken Statement for Ma'am:

"Slide 4 shows our four-entity system architecture: Data Owners (Hospital IoT gateways) encrypt medical files and generate keyword tokens. The Cloud Service Provider stores the heavy ciphertexts and performs matching over encrypted indices. The Ethereum Blockchain hosts our Result Verification Smart Contract (RVSC) and tracks document revocations on an immutable ledger. Finally, Data Users (Doctors/Nurses) query the cloud using blinded trapdoors and decrypt records using attribute private keys."

■ **Algorithm Mentioned:** Hybrid Off-Chain / On-Chain Split (Location: src/bamks_system.py:82–120 | contracts/BAMKS_Registry.sol:10–45)

What it Does: Directs heavy ciphertexts to cloud storage while pinning 256-bit tags (y_k) and status flags to smart contract storage.

■ Questions Ma'am Can Ask on this Slide & Direct Answers:

• **Q: Why not store files directly on the blockchain?**

Answer: A: Storing 1 MB on Ethereum costs thousands of dollars in gas fees, exceeds block gas limits (30M gas), and permanently publishes confidential health data on a public ledger violating HIPAA and GDPR.

• **Q: Who runs the blockchain?**

Answer: A: In production, a private consortium EVM (like Hyperledger Besu) run collaboratively by accredited healthcare providers.

SLIDE 5 OF 15: METHODOLOGY: ENCRYPTION & UPLOAD FLOW

■ ■ Dual-Screen Synchronization: Click on Doctor Prashant identity; show Patient Record #101 ciphertext hex, nonce, and GCM tag in the card.

■ ■ Exact Spoken Statement for Ma'am:

"Slide 5 details our upload pipeline. When a hospital encrypts a patient record: First, it derives a 256-bit symmetric key from token Φ via KDF. Second, AES-256-GCM encrypts the plaintext, producing ciphertext, a 96-bit nonce, and a 128-bit authentication tag. Third, it generates a discrete log public tag $y_k = g^{\sigma_k} \bmod P$. Fourth, it blinds the document's keywords into tokens $g^{(\eta * H1(kw))}$. Finally, in parallel: heavy ciphertexts go to the Cloud, while docId and public tag y_k are registered on-chain via our smart contract."

■ ■ Algorithm Mentioned: **file_encrypt & index_gen** (Location: src/bamks_system.py:82–119 | src/crypto_utils.py:26–34)

What it Does: Generates authenticated ciphertext payload, discrete log tag y_k , and discrete log keyword indices.

■ Questions Ma'am Can Ask on this Slide & Direct Answers:

• Q: How is the nonce generated?

Answer: A: Via PyCryptodome's `AES.new(key, AES.MODE_GCM)`, which calls the OS kernel CSPRNG (BCryptGenRandom on Windows). It is unique for every record to prevent ciphertext pattern leakage.

• Q: What is the purpose of public tag y_k ?

Answer: A: It is the on-chain anchor $y_k = g^{\sigma_k} \bmod P$ used later by the smart contract to verify the cloud's Schnorr Zero-Knowledge proof.

SLIDE 6 OF 15: METHODOLOGY: SEARCH & DECRYPTION FLOW (THE MASTER FLOWCHART)

■ ■ Dual-Screen Synchronization: Trigger a search for ['Cardiology', 'ECG']; show the real-time search latency, proof details, and decrypted text.

■ ■ Exact Spoken Statement for Ma'am:

"Slide 6 is our core operational flowchart. In the Search Phase: The doctor enters keywords, blinds them with random secret ϕ into trapdoor tokens ($T1, T2, T3$), and sends them to the cloud. The cloud tests trapdoors against the index using bilinear pairings without knowing the words. In the Blockchain & Decryption Phase: The cloud submits matched IDs to the RVSC smart contract. The contract checks `deletedDocRegistry` in $O(1)$ time. If active, the doctor's CP-ABE attribute key recovers symmetric secret Φ , and AES-256-GCM decrypts the record. If deleted, access is blocked immediately on-chain."

■ ■ Algorithm Mentioned: **trapdoor_gen & search_with_filter & aes_decrypt** (Location: src/bamks_system.py:121–157 | src/dynamic_extension.py:62–78)

What it Does: Blinds query into trapdoors, filters revoked IDs on-chain, and performs two-stage CP-ABE/AES decryption.

■ Questions Ma'am Can Ask on this Slide & Direct Answers:

• Q: How does the cloud search without knowing the keywords?

Answer: A: It checks algebraic equality: $e(T1, C1) * e(T2, C2) == e(T3, C3)$. The secret exponents in the trapdoor and index multiply in the target pairing group G_T without exposing the underlying words.

• Q: What happens if a revoked document is returned by a malicious cloud?

Answer: A: The RVSC contract runs `require(!deletedDocRegistry[id])`. The transaction reverts, the proof is rejected, and the doctor's device discards the result.

SLIDE 7 OF 15: ALGORITHMS USED: CRYPTOGRAPHIC PRIMITIVES BREAKDOWN

■ Dual-Screen

Synchronization: Show the Cryptographic Keys tab or modal with 256-bit prime P , generator g , and master public keys.

■ Exact Spoken Statement for Ma'am:

"Slide 7 summarizes the six core algorithmic primitives we implemented: 1. AES-256-GCM for hardware-accelerated, authenticated payload encryption. 2. CP-ABE for attribute-based policy trees. 3. Solidity Smart Contracts (BAMKS_Registry.sol) for on-chain state mapping. 4. Bilinear Pairing Search for conjunctive keyword matching. 5. Schnorr Non-Interactive Zero-Knowledge (SNIZK) proofs for result verifiability. 6. Inverted Keyword Index for sub-linear search lookup."

■ **Algorithm Mentioned:** **Cryptographic Primitives Suite** (Location: src/crypto_utils.py:1–89 | src/bamks_system.py:1–60)

What it Does: Defines cyclic group Z_p^* , hash-to-point H_1 , AES-256-GCM ciphers, bilinear pairing maps, and Schnorr proofs.

■ Questions Ma'am Can Ask on this Slide & Direct Answers:

• **Q: What is the security parameter?**

Answer: A: 256-bit prime field order Z_p^* providing 128-bit symmetric security level equivalent to NIST recommendations.

• **Q: Why GCM mode over CBC mode?**

Answer: A: CBC requires separate HMAC for integrity (Encrypt-then-MAC). GCM provides authenticated encryption with associated data (AEAD) in a single pass and prevents padding oracle attacks.

SLIDE 8 OF 15: BLOCKCHAIN ARCHITECTURE: ON-CHAIN VS OFF-CHAIN ALLOCATION

■ Dual-Screen

Synchronization: Switch to the Blockchain Ledger tab; show the transaction history with tx hashes, blocks, and gas used.

■ Exact Spoken Statement for Ma'am:

"Slide 8 illustrates our hybrid data tiering. We store heavy encrypted records, keyword tokens, and Schnorr proof witnesses off-chain in the cloud. On-chain, we store only lightweight 256-bit metadata: document IDs, public tags y_k , and the deletedDocRegistry boolean flags. This gives us the best of both worlds: zero storage bloat on Ethereum, but 100% tamper-proof auditability and $O(1)$ revocation enforcement."

■ **Algorithm Mentioned:** **registerDocument & deleteDocument** (Location: contracts/BAMKS_Registry.sol:50–77)

What it Does: Stores 4-slot metadata on registration and flips a single 32-byte storage slot on revocation.

■ Questions Ma'am Can Ask on this Slide & Direct Answers:

• **Q: What is the gas cost of deleteDocument?**

Answer: A: Exactly 42,100 gas units (21,000 base tx + 20,000 SSTORE opcode + 1,100 LOG event).

• **Q: Why is mapping lookup $O(1)$ in Solidity?**

Answer: A: Solidity mappings use Keccak-256 hashing to calculate deterministic 32-byte storage slots ($\text{keccak256}(\text{key} \cdot \text{slot})$). Lookup is constant time $O(1)$ regardless of mapping size.

SLIDE 9 OF 15: CP-ABE ACCESS CONTROL: FINE-GRAINED POLICY TREES

■ ■ Dual-Screen Synchronization: LIVE DEMO: Switch between Dr. Prashant (Cardiology) and Dr. Rushikesh (Neurology). Show how doc cards flip between AUTHORIZED and POLICY_DENIED.

■ ■ Exact Spoken Statement for Ma'am:

"Slide 9 explains our access control engine. In BAMKS-D, patient records are governed by policy trees such as (Role_Doctor AND Dept_Cardiology). On the right screen, observe our live demonstration: When Dr. Prashant is logged in, cardiology cases #101 and #102 decrypt into plain English, while neurology cases #103 and #104 remain cryptographically locked. When we switch to Dr. Rushikesh, his neurology credentials immediately unlock #103 and #104, while locking cardiology cases. If Nurse Anita logs in, only routine pediatric records can be decrypted."

■ ■ Algorithm Mentioned: **check_policy & token_encrypt_onchain** (Location: app.py:121–135 | src/bamks_system.py:100–106)

What it Does: Evaluates policy clauses and binds master key Phi to ciphertext components C and C0.

■ Questions Ma'am Can Ask on this Slide & Direct Answers:

• Q: What prevents Dr. Rushikesh and Nurse Anita from pooling keys (collusion)?

Answer: A: Each user's private key contains unique random polynomials tied to their Global Identifier (UID). Pooling mismatched keys produces random exponents that fail bilinear reconstruction.

• Q: Can the cloud decrypt the file if it has the policy?

Answer: A: No. The cloud does not possess any user private keys (SK_uid) issued by the Attribute Authority.

SLIDE 10 OF 15: O(1) REVOCATION MECHANISM: TRADITIONAL RE-KEYING VS BAMKS-D

■ ■ Dual-Screen Synchronization: Click 'Delete Doc #101'; show the console output: Execution time 0.0001 ms, EVM tx logged, and status changed to REVOKED.

■ ■ Exact Spoken Statement for Ma'am:

"Slide 10 demonstrates our flagship research breakthrough: O(1) revocation. In traditional schemes, revoking Doc #101 required downloading the database, generating a new master key, re-encrypting all L files, and re-uploading everything. In BAMKS-D, revoking a document requires sending one lightweight transaction calling deleteDocument(101). The smart contract sets deletedDocRegistry[101] = true in constant time (0.001 ms). Notice on our live portal: Doc #101 was revoked in under a millisecond and is now permanently excluded from search results!"

■ ■ Algorithm Mentioned: **SingleDocDelete** (Location: src/dynamic_extension.py:36–48 | contracts/BAMKS_Registry.sol:69–77)

What it Does: Executes O(1) state write to EVM storage slot without touching cloud ciphertexts or master keys.

■ Questions Ma'am Can Ask on this Slide & Direct Answers:

• Q: Is the revoked file still physically in cloud storage?

Answer: A: Yes, but it is unusable ciphertext. The smart contract RVSC rejects any search results containing its ID, and key derivation excludes it. Physical cleanup can be handled asynchronously.

• Q: Can an attacker undo the deletion?

Answer: A: No. The deleteDocument function is protected by the onlyDocOwner modifier; only the authenticated document owner or hospital admin can revoke records.

SLIDE 11 OF 15: KEY FEATURES & INNOVATIONS: SUMMARY OF SYSTEM CAPABILITIES

■ Dual-Screen

Synchronization: Stay on live dashboard; show the 6 feature summary cards in the UI.

■ Exact Spoken Statement for Ma'am:

"Slide 11 synthesizes our six primary engineering innovations: 1. $O(m)$ single-document addition without dataset re-indexing. 2. $O(1)$ instant on-chain revocation mapping. 3. End-to-end zero-plaintext leakage to the cloud. 4. Multi-keyword conjunctive trapdoor searches. 5. Tamper-proof blockchain audit trails. 6. Cryptographically sound zero-knowledge result verification."

■ Algorithm Mentioned: **SingleDocModify (Bonus Algorithm)** (Location: src/dynamic_extension.py:50–59)

What it Does: Implements single-document modification as an atomic delete followed by single-document insert.

■ Questions Ma'am Can Ask on this Slide & Direct Answers:

• Q: How does document modification work?

Answer: A: As defined in Section 6 of our Review 1 report, modification is executed as `SingleDocDelete(old_doc)` followed by `SingleDocAdd(new_doc)`, taking $O(m)$ time instead of $O(L \times m)$.

• Q: What is a conjunctive query?

Answer: A: A query where the result must match ALL specified keywords (e.g., 'Cardiology' AND 'ECG' AND 'Emergency').

SLIDE 12 OF 15: PERFORMANCE COMPARISON: EMPIRICAL BENCHMARK RESULTS

■ Dual-Screen

Synchronization: Navigate to the Benchmark Lab on the dashboard; click 'Run Real-Time Benchmark' and show the live speedup factor.

■ Exact Spoken Statement for Ma'am:

"Slide 12 presents our rigorous empirical benchmark data comparing the base paper against BAMKS-D. For document addition, the base paper required 5,560 ms, while our `SingleDocAdd` executes in 0.12 ms — a 46,000x speedup. For revocation, our $O(1)$ blockchain flag executes in 0.001 ms compared to 5,560 ms. For search, our system adds a negligible 6% overhead (340 ms vs 320 ms) solely to execute the on-chain zero-knowledge audit. Notice on our live benchmark widget: running actual Python exponentiations confirms this exact speedup factor live!"

■ Algorithm Mentioned: **live_benchmark** (Location: app.py:560–602 | src/dynamic_extension.py:80–101)

What it Does: Runs real Python BigInt modular exponentiations comparing $L \cdot m$ operations against $1 \cdot m$ operations.

■ Questions Ma'am Can Ask on this Slide & Direct Answers:

• Q: What causes the 6% search overhead?

Answer: A: The cloud computing the Schnorr commitment R and aggregated proof $\pi_{\hat{}}$, plus the smart contract executing `verifyResultProof`.

• Q: Does addition time depend on the size of the database?

Answer: A: No! Addition depends strictly on m (the number of keywords in that single file, typically 5–10), completely independent of total files L in the hospital.

SLIDE 13 OF 15: TESTING & VALIDATION: SECURITY AND INTEGRITY RESULTS

■ ■ Dual-Screen Synchronization: Show the terminal execution logs or the Security Verification card on the dashboard showing 100% test pass rate.

■ ■ Exact Spoken Statement for Ma'am:

"Slide 13 summarizes our testing and validation across six security modules: CP-ABE access authorization, keyword trapdoor blinding, Schnorr ZKP proof validity, on-chain revocation enforcement, replay attack immunity, and GCM authentication tag verification. All unit tests passed with 100% integrity, confirming zero ciphertext leakage and zero unauthorized decryptability across our simulated clinical scenarios."

■ ■ Algorithm Mentioned: **test_demo.py test suite** (Location: test_demo.py:1–180 | review2_demo.py:1–200)

What it Does: Automated verification suite validating cryptographic correctness and access control edge cases.

■ Questions Ma'am Can Ask on this Slide & Direct Answers:

• Q: How did you test replay attacks?

Answer: A: We verified that search trapdoors incorporate fresh random numbers `phi_rand` and timestamps; replaying old trapdoors does not allow correlating subsequent searches.

• Q: What happens if someone modifies 1 bit of ciphertext in cloud storage?

Answer: A: AES-256-GCM `decrypt_and_verify` throws a `MAC check failed exception` immediately, preventing corrupted medical records from being read.

SLIDE 14 OF 15: IMPLEMENTATION & TECH STACK: FULL ARCHITECTURE STACK

■ ■ Dual-Screen Synchronization: Open VS Code briefly to show the project structure: `app.py`, `src/dynamic_extension.py`, `contracts/BAMKS_Registry.sol`.

■ ■ Exact Spoken Statement for Ma'am:

"Slide 14 outlines our full engineering stack. The backend runs on Python 3.13 and Flask, utilizing PyCryptodome for AES-256-GCM and hashlib/secrets for high-entropy key generation. The smart contracts are written in Solidity (v0.8.20) for EVM compatibility. The frontend is built with vanilla HTML5, CSS3, and JavaScript, providing live role-based identity switching, EVM ledger telemetry, and cryptographic console inspection."

■ ■ Algorithm Mentioned: **Full Stack REST API** (Location: `app.py`:1–615 | `index.html`:1–2500)

What it Does: Exposes endpoints for `/api/status`, `/api/documents`, `/api/search`, `/api/add`, `/api/delete`, `/api/blockchain`.

■ Questions Ma'am Can Ask on this Slide & Direct Answers:

• Q: What Solidity compiler version is used?

Answer: A: Solidity ^0.8.20, utilizing modern memory management and custom error handling.

• Q: How do Python and Solidity communicate in your demo?

Answer: A: In our Review 2 prototype, the Flask backend executes the EVM state machine simulating storage slot mutations and gas tracking; for Review 3, it connects via `Web3.py` to Sepolia.

SLIDE 15 OF 15: CONCLUSION & REVIEW 3 ROADMAP: FINAL DELIVERABLES

■ ■ Dual-Screen

Synchronization:

Point to the Roadmap card on the dashboard outlining Sepolia deployment and IoT gateway profiling.

■ ■ Exact Spoken Statement for Ma'am:

"To conclude our Review 2 presentation: We have successfully implemented the methodology committed in Review 1, solved the Section 8 open problem from Cheng et al. (Elsevier 2026), and achieved 46,000x faster updates with $O(1)$ on-chain revocation. For Review 3 (Final Review), as per the syllabus, we will: 1. Deploy our Solidity smart contract to the public Ethereum Sepolia testnet with verified Etherscan links. 2. Scale benchmarks to 10,000 real medical records. 3. Profile CPU and battery power consumption on physical Raspberry Pi IoT hardware. 4. Complete the final thesis report. Thank you Ma'am, we are ready for your questions!"

■ ■ Algorithm Mentioned:

Review 3 Experimental Plan (Location: REVIEW_2_PRESENTATION_SCRIPT.md:60–67)

What it Does:

Outlines testnet migration, zk-SNARK optimization, and hardware energy profiling roadmap.

■ Questions Ma'am Can Ask on this Slide & Direct Answers:

- Q: What will be the primary deliverable for Review 3?
Answer: A: Migration to public Ethereum Sepolia testnet with live transaction auditing, physical Raspberry Pi power profiling, and the complete project report.
- Q: Can this be deployed in real hospitals today?
Answer: A: Yes, by hosting the smart contract on an accredited private consortium EVM (like Hyperledger Besu) and pairing with hospital PACS/EHR servers.

SECTION 4: TOP 20 MASTER VIVA DEFENSE QUESTIONS & ANSWERS

1. What exact research gap in Cheng et al. did you solve?

In Section 8 of Cheng et al. (Elsevier 2026), the authors explicitly stated that BAMKS is restricted to whole-dataset versions, requiring $O(L \times m)$ re-encryption of all L files upon adding or deleting a single file. We solved this by decoupling file indices, achieving $O(m)$ single-document addition and $O(1)$ on-chain revocation.

2. Why is deletion $O(1)$ complexity and how is it proven?

Instead of modifying cloud ciphertexts, deletion updates an on-chain mapping: `deletedDocRegistry[docId] = true`. In the EVM, writing to a mapping executes the `SSTORE` opcode, which computes a Keccak-256 hash of the key and slot to write to a single 32-byte storage slot in constant $O(1)$ time regardless of whether the database has 10 records or 10 million records.

3. Is the gas cost real or just a demo number?

It is 100% mathematically real according to the official Ethereum Yellow Paper EVM Opcode specification: 21,000 gas (base transaction fee) + 20,000 gas (`SSTORE` opcode to set a zero slot to non-zero) + 1,100 gas (`LOG2` event emission) = 42,100 gas units.

4. If we don't pay money, why are we using Gwei and Blockchain?

No enterprise software developer uses real Ethereum Mainnet dollars during development; we use local EVM nodes and testnets where test ETH is free, but the EVM opcode execution rules and gas metering are identical. Furthermore, hospital consortia deploy on Private Consortium EVMs (Hyperledger Besu) where gas price is set to 0 Gwei (free), using gas purely for rate-limiting and DoS prevention.

5. How are nonces generated and why are they needed in AES-256-GCM?

In `src/crypto_utils.py` line 28, `PyCryptodome` pulls a 128-bit random nonce from the OS kernel CSPRNG (`BCryptGenRandom`). In GCM mode, reusing a nonce with the same key breaks confidentiality. A fresh nonce ensures that even if two patients have identical medical reports, their ciphertexts look completely distinct.

6. Where does Bilinear Pairing happen in your system?

It happens in two places: 1. During Global Setup by KGC and Attribute Authorities to compute master parameters like $e(g,g)^\mu$ (`src/bamks_system.py:28`). 2. Locally on the doctor's client device during CP-ABE decryption to pair ciphertext component $C_0 = g^s$ with attribute keys K_1, K_2 to unlock master secret token Φ .

7. If the cloud generates a fresh random token every search, how does RVSC verify it?

RVSC does not check for a static password; it verifies an invariant algebraic equation: $R == g^{\pi} * \prod (y_k^{h_k}) \bmod P$. When expanded, the secret exponent σ_k cancels out with the on-chain public tag $y_k = g^{\sigma_k}$. If the cloud searched honest records, the exponents cancel out to R regardless of the random blinding numbers chosen.

8. What prevents an offline keyword guessing attack (KGA)?

Keywords are blinded inside search trapdoors using a fresh, secret ephemeral random exponent ϕ_{rand} known only to the doctor: $T_3 = g^{(\phi_{\text{rand}} * \sum(H_1(kw)))}$. Without knowing ϕ_{rand} or user private keys, an untrusted cloud cannot test candidate words from a medical dictionary.

9. What is CP-ABE collusion resistance?

It guarantees that two unauthorized users (e.g. Dr. Rushikesh with `Dept=Neurology` and Nurse Anita with `Role=Nurse`) cannot pool their private keys to open a record requiring '`Role=Doctor AND Dept=Cardiology`'. Each user's private key components are personalized with their unique Global Identifier (UID) using independent random polynomials that cannot mathematically combine.

10. Why did you use AES-256-GCM alongside CP-ABE?

CP-ABE relies on elliptic curve pairings, which are computationally heavy for multi-megabyte medical records. We follow the standard KEM-DEM hybrid model: CP-ABE securely encapsulates the small 256-bit symmetric key Φ , while AES-256-GCM handles fast, authenticated payload encryption.

11. What is the cloud's threat model?

The cloud is semi-honest (honest-but-curious) — it follows the protocol to store files and run queries, but tries to infer patient diagnosis from ciphertexts and search patterns. Furthermore, we protect against lazy clouds that return incomplete results by having RVSC audit the Schnorr Zero-Knowledge proof.

12. What happens if a revoked file is still in cloud storage?

The ciphertext in cloud storage remains encrypted with AES-256-GCM and is undecryptable without the key. More importantly, the RVSC smart contract enforces `require(!deletedDocRegistry[id]);` if the cloud attempts to return a deleted document, the on-chain verification reverts and the result is discarded.

13. How does BAMKS-D protect against replay attacks?

Each search trapdoor includes a fresh random blinding factor `phi_rand` and timestamp. Replaying an old trapdoor will not decrypt subsequent search results, and old tokens cannot be linked to the user's ongoing query profile.

14. How does document modification work in your code?

In `src/dynamic_extension.py` line 50, `SingleDocModify` is implemented as an atomic sequence: `SingleDocDelete(old_doc_id)` followed by `SingleDocAdd(new_doc_id)`, completing in $O(m)$ time without touching other files in the database.

15. What are the roles of Prashant Singh and Adak Rushikesh?

Prashant Singh (24BYB1042) implemented the core 256-bit cryptographic engine, AES-256-GCM authenticated pipeline, trapdoor generation, and Python REST API. Adak Rushikesh (24BYB1055) developed the Solidity smart contract (`BAMKS_Registry.sol`), the on-chain `deletedDocRegistry` mapping, and the EVM gas benchmarks.

16. What is the difference between CP-ABE and KP-ABE?

In KP-ABE (Key-Policy), the policy is inside the user's key and ciphertexts have attributes. In CP-ABE (Ciphertext-Policy), the user's key has attributes and the data owner attaches the policy tree to the ciphertext. Healthcare requires CP-ABE because the patient/hospital must dictate who accesses their records.

17. What is an authentication tag in AES-GCM?

It is a 128-bit cryptographic MAC generated using Galois field multiplication over the ciphertext and nonce. If an attacker or untrusted cloud flips even a single bit of the stored ciphertext, `decrypt_and_verify` throws an exception and halts decryption.

18. What is the order of cyclic group G ?

A 256-bit prime $P = 0xFFC2F$ (secp256k1 base field order), ensuring discrete logarithm hardness.

19. Why does single-document addition take $O(m)$ time?

Because keyword tokens $g^{(\eta * H1(kw))}$ are calculated ONLY for the m keywords belonging to the newly inserted document. It does not require re-encrypting or re-indexing any of the other $L-1$ existing documents.

20. What is your roadmap for Review 3?

1. Deploying the smart contract to public Ethereum Sepolia testnet with live Etherscan links. 2. Scaling benchmarks from 50 to 10,000 real medical records. 3. Physical hardware CPU and battery power profiling on a Raspberry Pi IoT gateway. 4. Final thesis document submission.